

Seminar 7 - Ansamblu

1. Se consideră un TAD FluxDeDate care conține numere reale și care are următoarele operații:
 - creează(flux): construiește un container vid
 - adaugă(flux, elem): adaugă un nou element
 - mediana(flux): returnează mediana elementelor din fluxul de date curent.Dacă șirul curent conține un număr impar de elemente, mediana reprezintă numărul din mijlocul șirului format prin ordonarea elementelor din șirul inițial. Dacă șirul curent conține un număr par de elemente, mediana este media celor două numere aflate pe pozițiile din mijloc în șirul ordonat.

Să se implementeze operațiile și să se menționeze complexitatea lor.

Exemplu:

conținutul containerului la un moment dat: 6, 1, 12 => mediana este 6
se adauga 13 => mediana este $(6+12)/2=9$
se adaugă 20 => mediana este 12
se adauga -3 => mediana este 9

Metoda 1: memorăm elementele în ordine crescătoare într-un vector. Astfel, mediana se va putea obține eficient folosind poziția/pozițiile din mijlocul vectorului ordonat.

Complexitate adaugare: $O(n)$

Complexitate calcul mediana: $\Theta(1)$

Metoda 2: Separăm datele în două ansambluri care conțin un număr (aproximativ) egal de elemente, cu proprietatea ca elementele din primul ansamblu sunt mai mici sau egale decât elementele din cel de-al doilea ansamblu. Dacă șirul conține un număr impar de elemente, $2k+1$, vom presupune în continuare că memorăm în primul ansamblu $k+1$ elemente, iar în al doilea k .

Elementele necesare pentru calculul mediane vor fi: cel mai mare element din primul ansamblu și cel mai mic element din al doilea ansamblu. Pentru a putea accesa aceste elemente în timp constant, vom folosi pentru memorarea elementelor mai mici un ansamblu maximal, iar pentru memorarea elementelor mai mari - un ansamblu minimal.

Operația de calcul a mediane:

- dacă numărul total de elemente este impar => mediana este rădăcina primului ansamblu
- în caz contrar => mediana este media rădăcinilor celor două ansambluri

Operației de adaugare a unui element:

- o idee posibilă de implementare constă în a identifica în care dintre cele două ansambluri trebuie adăugat elementul
- dacă șirul este vid sau dacă elementul curent este mai mic sau egal decât rădăcina primului ansamblu, adăugăm elementul în acest ansamblu. În caz contrar, vom adăuga elementul în al doilea ansamblu.
- dacă în urma adăugării, primul ansamblu va avea cu 2 elemente mai mult decât cel de-al doilea, trebuie să mutăm un element din primul ansamblu în cel de-al doilea. Element care poate fi mutat este elementul maxim. În mod similar, dacă al doilea ansamblu are mai multe elemente decât primul, vom muta elementul lui minim în primul ansamblu.

Reprezentare:

FluxDeDate:

```
elem_mici: Ansamblu //ansamblu maximal  
elem_mari: Ansamblu //ansamblu minimal
```

Pentru Ansamblu vom presupune că avem următoarele operații (implementarea lor poate fi consultată în Cursul 14):

- creeaza(ansamblu, relatie)
- adauga(ansamblu, elem)
- prim(ansamblu): returnează valoarea aflată în rădăcina ansamblului (primul element din vector), care reprezintă elementul maxim dacă ansamblu este maximal, respectiv elementul minim dacă ansamblul este unul minimal
- sterge(ansamblu): sterge și returnează primul element al ansamblului (rădăcina)
- dim(ansamblu): returnează numărul de elemente din ansamblu

subalgorithm creeaza(flux) este:

```
creeaza(flux.elem_mici, "≥")  
creeaza(flux.elem_mari, "≤")
```

sf_subalgorithm

Complexitate: $\theta(1)$

subalgorithm mediana(flux) este:

```
daca dim(flux.elem_mici) = 0 atunci:
```

```
@eroare, mediana nu poate fi calculata
```

```
sf_daca
```

```
daca dim(flux.elem_mici) = dim(flux.elem_mari) atunci:
```

```
mediana ← (prim(flux.elem_mici)+prim(flux.elem_mari))/2
```

```

    altfel:
        mediana ← prim(flux.elem_mici)
    sf_daca
sf_subalgoritm

```

Complexitate: $\theta(1)$

```

subalgoritm adauga(flux, elem) este:
    daca dim(flux.elem_mici) = 0 atunci:
        adauga(flux.elem_mici, elem)
    altfel
        daca elem ≤ prim(flux.elem_mici) atunci:
            adauga(flux.elem_mici, elem)
        altfel
            adauga(flux.elem_mari, elem)
    sf_daca

    daca dim(flux.elem_mici) > dim(flux.elem_mari)+1 atunci:
        r ← sterge(flux.elem_mici)
        adauga(flux.elem_mari, r)
    altfel daca dim(flux.elem_mici) < dim(flux.elem_mari)
atunci:
        r ← sterge(flux.elem_mari)
        adauga(flux.elem_mici, r)
    sf_daca

    sf_daca
sf_subalgoritm

```

Complexitate: $O(\log n)$

2. Se dau m liste nevide de numere întregi ordonate crescător. Să se determine intervalul de lungime minimă $[a, b]$, unde $a, b \in \mathbb{Z}$, care conține cel puțin un element din fiecare listă. Dacă există 2 intervale $[a, b]$ și $[c, d]$ de aceeași dimensiune, să se returneze intervalul $[a, b]$ dacă $a < c$.

Exemplu:

L1: 4 -> 7 -> 9 -> 13 -> 15

L2: 0 -> 8 -> 10 -> 14

L3: 6 -> 12 -> 16 -> 30 -> 50 -> 80

Intervalul de lungime minimă este $[6, 8]$

Idee de rezolvare: vom considera pentru început elementele aflate pe prima poziție în cele m liste și vom calcula valoarea minimă și maximă. Notăm aceste valori cu min , respectiv max . Intervalul obținut, $[min, max]$, conține câte un element din fiecare lista. Pentru a căuta un interval mai mic, vom avansa în lista din care a provenit elementul minim (înlocuind elementul minim curent cu elementul următor din listă). În colecția nou obținută de m elemente, vom calcula din nou minimul și maximul și vom compara lungimea intervalului obținut cu lungimea intervalului optim identificat până în prezent. Atunci când epuizăm elementele dintr-o listă, vom opri parcurgerea fiindcă avansarea în altă listă decât cea a minimului nu va putea duce la un interval de lungime mai mică.

- În ce structura de date putem memora cele m pozitii curente din fiecare listă?

Opțiunea 1: memorăm pozițiile elementelor curente într-o listă/într-un vector. În acest caz, calculul minimului va avea complexitatea $\theta(m)$. Pentru a actualiza elementul maxim este suficient să comparăm noul element cu maximul curent. În cel mai rău caz, se va repeta procesul de recalculare a minimului și maximului pentru fiecare element din fiecare listă (cu excepția elementelor de pe prima poziție). Astfel, dacă notăm cu n numărul total de elemente din cele m liste, obținem complexitatea $O(m * n)$.

Opțiunea 2: memorăm elementele într-un ansamblu minimal pentru a eficientiza calculul minimului..

Vom presupune că datele de intrare sunt memorate în m liste înlănțuite, formate din noduri cu următoarea reprezentare:

<u>Nod:</u>	<u>Lista:</u>
e: TElem	prim: ↑Nod
urm: ↑Nod	

functie intervalMinim(liste, m) **este:**

```
// liste este un vector continand m liste
creeaza(a, "≤")// vom memora in ansamblu nodurile pentru a
putea avansa la urmatorul element din lista; relatia de ordine va
compara valorile din noduri
```

```
max_val ← [liste[0].prim].e
pentru i ← 0, m-1 execută:
    adauga(a, liste[i].prim)
    max_val ← max(max_val, [liste[i].prim].e)
sf_pentru
min_val ← [prim(a)].e
```

```

gata ← fals
max_curent ← max_val
cât timp ¬ gata execută:
    nod ← sterge(a) // eliminăm elementul minim
    dacă [nod].urm = NIL atunci:
        gata ← adevărat
    altfel
        // adăugăm nodul următor din lista minimului
        adauga(a, [nod].urm)
        // actualizăm maximumul și minimumul
        max_curent ← max(max_curent, [[nod].urm].e)
        min_curent ← [prim(a)].e // minimumul calculat
folosind nodurile din ansamblu
    dacă max_curent - min_curent < max_val - min_val atunci:
        max_val ← max_curent
        min_val ← min_curent
    sf_dacă
sf_dacă
sf_cât timp
    interval_minim ← <min_val, max_val>
sf_subalgoritm

```

Complexitate: $O(n \log k)$ (în cel mai rău caz vom ajunge să adăugăm toate elementele din toate listele în ansamblu)